



Performance Testing

Date	2 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well Being
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter 5.6
Type of Testing	Load Testing and Stress Testing
Target Module / API	/predict endpoint - Well-being prediction API (accepts life expectancy, mean years of schooling, expected years of schooling, and GNI per capita; returns predicted well-being category)
Test Environment	Cloud / Hosted Server (Render deployment)
Test Date	4 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Predict HDI for a single country with valid inputs (baseline sanity check)	1	10	API returns correct HDI category within 2 seconds
2	Concurrent users submitting Scenario 1 type inputs (Very High HDI countries)	50	60	All requests return Very High HDI classification with response time < 2 seconds
3	Load test simulating policymakers submitting Scenario 2 type mid-range inputs	100	120	System sustains 100 concurrent users with error rate below 1%
4	Stress test with peak load simulating Scenario 3 type low-indicator inputs	200	180	System remains stable or degrades gracefully without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 seconds	Pass	Well within target for HDI prediction calls
2	Response Time (Max)	< 5 seconds	3.4 seconds	Pass	Observed during peak of 200 virtual users
3	Throughput (Req/sec)	N/A	68 req/sec	Pass	Stable throughput up to 100 concurrent users
4	Error Rate	< 1%	0.3%	Pass	Minor timeouts only under 200-user stress test
5	CPU Utilization	< 80%	74%	Pass	Peaked during stress test scenario
6	Memory Utilization	< 80%	69%	Pass	Model and scaler kept in memory; stable across runs

Step 4: Observations & Analysis

Key Findings:

The HDI prediction API consistently classified countries correctly across all three scenarios (Very High, Medium, and Low HDI) under normal and elevated load. Average response times stayed under 1.5 seconds up to 100 concurrent users, and the system degraded gracefully rather than failing outright at the 200-user stress level.

Bottlenecks Identified:

A minor increase in response time (up to 3.4 seconds) was observed beyond 150 concurrent users, traced to model inference on a single-threaded Flask development server. CPU utilization also approached 74% during the stress test, indicating limited headroom on the free-tier hosting instance.

Optimization Steps Taken:

The Flask app was switched to run behind Gunicorn with multiple worker processes, the trained model and scaler were cached in memory at startup instead of being reloaded per request, and input validation was optimized to reduce preprocessing overhead. These changes reduced average response time by approximately 35% during re-testing.

Step 5: Screenshots / Evidence

Screenshot 1: JMeter Summary Report showing average, min, and max response times, throughput, and error % across all four test scenarios (file: jmeter_summary_report.png)

Screenshot 2: JMeter Response Time Graph and Active Threads Over Time chart for the 200-user stress test scenario (file: jmeter_response_time_graph.png)